

Лабораторная работа №3. Управление вращением робота

1. Цель работы

Реализация механизма отправки и приема сообщений по последовательному порту. Интерпретация полученных сообщений и преобразование их в углы поворота в средах разработки Unity и Arduino IDE. Синхронизация поворотов виртуального и реального роботов с использованием потенциометров.

2. Порядок выполнения

- Установить среду разработки Arduino IDE, драйвер UART и необходимые библиотеки.
- Протестировать работу сервоприводов, потенциометров и дисплея с помощью готовых скетчей.
- Реализовать передачу данных с робота в Unity по последовательному порту. Добавить в сцену Unity интерфейс работы с последовательным портом.
- Написать программу для приёма строки данных в Unity и парсинга её для получения углов поворота.
- Синхронизировать вращение модели робота в Unity с реальным роботом на основе данных, получаемых с потенциометров.
- Реализовать передачу данных об углах поворота робота из Unity на микроконтроллер, приём и парсинг данных на стороне микроконтроллера.
- Добавить возможность поворачивать реального робота на углы, полученные из Unity.
- Продемонстрировать проект преподавателю, ответить на вопросы.
- Оформить отчёт и загрузить его в ЛК (в PDF).

3. Варианты задания

Поскольку количество физических экземпляров роботов ограничено (10 шт.), студентам предлагается объединиться в группы по 2 человека. Один человек реализует передачу данных и управление в одну сторону (с робота в Unity), другой – в другую (из Unity на робота).

4. Проверка работы оборудования

Откройте тестовый скетч «**Unpacking_test_code.ino**». По умолчанию он расположен по адресу:

```
../ADA031-Adeept_Robotic_Arm_Kit_for_Arduino-V4.0-20240326\Code\_5  
Unpacking_test_code\Unpacking_test_code
```

Выполните его проверку/компиляцию (**Ctrl + R**). В случае успешной компиляции загрузите скетч на плату (**Ctrl + U**).

4.1. Проверка сервоприводов

Режим проверки сервоприводов задаётся целочисленной переменной `control`. Если `control = 1`, все сервоприводы будут проверяться автоматически, последовательно один из другим. Если `control = 0`, то для переключения активного в данный момент сервопривода нужно будет нажать клавишу «**Button**» на плате.

```
// Поочередно определить все пины.  
int pin[] = {9, 6, 5, 3, 11, 10};  
// Количество пинов для проверки.  
const int PIN_COUNT = sizeof(pin) / sizeof(pin[0]);  
// Создать объекты для проверки сервоприводов.  
Servo s[PIN_COUNT];  
// Режим проверки: 0 – независимый режим; 1 – полный контроль.  
int control = 0;
```

Во время проверки в консоль будут выводиться сообщения вида:

```
Individually attached: 2 (Pin:6) Control Model: 0, PIN_COUNT: 6
```

Будьте осторожны, сервоприводы склонны к перегреву!

4.2. Проверка потенциометров

Через 3 секунды после включения надпись "OLED OK!" исчезнет, и вместо неё на экране отобразятся "сырые" значения, считываемые с потенциометров в реальном времени (**не градусы**). Например, надпись вида «a:0_499, 1_492» означает, что угол A0 равен 499, а угол A2 равен 485 (рисунок 4).



Рисунок 4. Данные с потенциометров об углах поворота

Для перевода этих значений в градусы используются две функции:

- **analogRead.** Считывает "сырое" аналоговое напряжение с контакта (например, A0) и преобразует его в цифровое число в диапазоне от 0 до 1023.
- **map.** Преобразует (т.е. масштабирует) это число из диапазона (0-1023) в новый диапазон. Результат записывается в целочисленную переменную. Для потенциометра A6, отвечающего за хват, доступны значения от 35° до 90°. Для остальных потенциометров – от 0° до 180°.

```
val1 = map(analogRead(0), 0, 1023, 0, 180);
```

Полученное значение в градусах затем можно вывести в консоль командой:

```
Serial.print("A0:");Serial.println(val1);
```

Примечание: манипулятор Adept использует 180-градусные позиционные сервоприводы с механическим редуктором, который позволяет вручную проворачивать вал при отсутствии питания. Диапазон вращения сервоприводов составляет от 0° до 180°.

5. Передача данных из Arduino IDE

Откройте скетч «potentiometer.ino» в Arduino IDE. Этот скетч читает аналоговые напряжения с потенциометров, преобразует их в углы из диапазона от 0 до 180 и выводит информацию в виде текста построчно по последовательному порту.

Изначально потенциометры тут обозначены так же, как на печатной плате: A0, A1, A2, A3, A6. Чтобы минимизировать количество передаваемого по последовательному интерфейсу текста, а также упростить будущий процесс парсинга строки упростим данный скетч.

```
void loop()
{
    val1 = map(analogRead(0), 0, 1023, 0, 180);
    val2 = map(analogRead(1), 0, 1023, 0, 180);
    val3 = map(analogRead(2), 0, 1023, 0, 180);
    val4 = map(analogRead(3), 0, 1023, 0, 180);
    val5 = map(analogRead(6), 0, 1023, 35, 90);
    Serial.print("A");Serial.print(val1); // A = A0
    Serial.print("B");Serial.print(val2); // B = A1
    Serial.print("C");Serial.print(val3); // C = A2
    Serial.print("D");Serial.print(val4); // D = A3
    Serial.print("E");Serial.print(val5); // E = A6
    Serial.println(";");
    delay(1000);
}
```

Пусть все данные выводятся в одну длинную строку без пробелов или разделительных символов с интервалом в 1000 мс (1 секунду). Вместо строки со звёздочками, игравшей роль разделителя, добавьте **символ конца строки – точку с запятой (;)**.

Загрузите изменённый скетч в микроконтроллер. В окне консольного вывода будет отображаться:

```
A85B84C92D87E60
```

6. Получение данных по последовательному порту в Unity

Логика работы в Unity с последовательным портом следующая:

- 1) Задать имя и скорость порта, инициировать соединение;
- 2) Открыть порт для прослушки;
- 3) Получить строку данных в текстовом виде, записать её в переменную типа TMP_Text. Всего 5 переменных: по 1 на каждый потенциометр;

- 4) Отобразить полученную информацию в отладчике и на экране;
- 5) Очистить переменные;
- 6) Закрыть порт.

Для работы с последовательным портом в C#-скрипт Unity нужно подключить библиотеку:

```
using System.IO.Ports;
```

Но чтобы отладчик "увидел" `System.IO.Ports`, потребуется внести изменения в настройки проекта. Перейдите в меню **«Project Settings >> Player >> Other Settings»** и параметр **«Api Compatibility Level*»** измените на **«.NET Framework»** (рисунок 5).

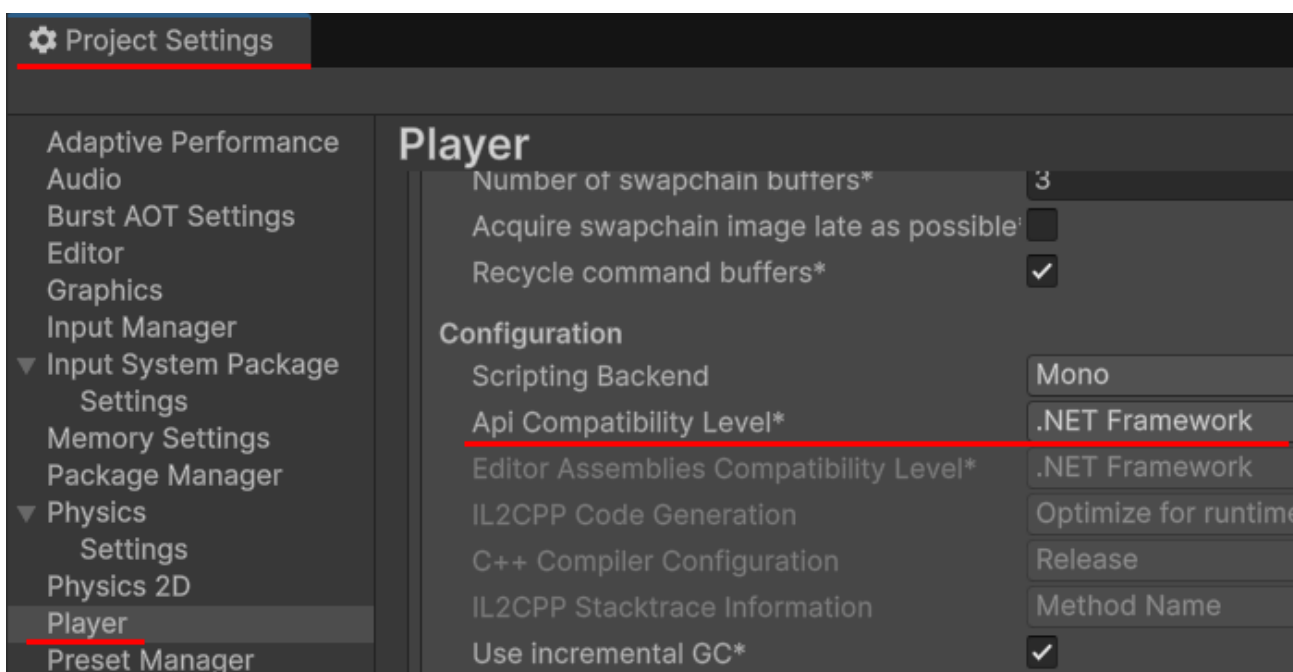


Рисунок 5. Изменение настроек проекта для совместимости

❖ Инициализация последовательного порта

Для работы с последовательным портом в C# (среде .NET) первым делом понадобится объявить переменную типа `System.IO.Ports.SerialPort` и задать номер виртуального COM-порта (такой же, как в Arduino IDE).

```
private System.IO.Ports.SerialPort serialPort;  
public string portName = "COM5";
```

Далее в методе `Start()` нужно инициировать порт, открыть его для получения сообщений и указать символ конца строки (пусть это будет `;`).

```

serialPort = new SerialPort(portName, 9600); // Название и скорость порта
serialPort.NewLine = ";"; // Символ конца строки
serialPort.ReadTimeout = 1500; // Таймаут в мс
try
{
    serialPort.Open(); // Открыть порт
}
catch
{
    Debug.Log("Ошибка открытия COM-порта");
}

```

Скорость передачи – **9600** бод (изменений сигнала в секунду).

Для получения данных нужно добавить задержку, чтобы данные успели прийти по COM-порту. В Unity это делается через сопрограмму и интерфейс `IEnumerator`. Задержка указывается в секундах как число типа `float`.

```

public void GetData()
{
    StartCoroutine(DelayedAction(1f)); // Задержка, чтобы данные успели
    // прийти по COM-порту
}
private IEnumerator DelayedAction(float delayInSeconds)
{
    try
    {
        yield return new WaitForSeconds(delayInSeconds);
        outputLine = serialPort.ReadLine();
        Debug.Log(outputLine);
        float[] result = ParseString(outputLine); // Запись в массив
        for (int i = 0; i <= 4; i++)
        {
            // Вывод элементов массива на интерфейс
            a0.text = result[0].ToString();
            a1.text = result[1].ToString();
            a2.text = result[2].ToString();
            a3.text = result[3].ToString();
            a6.text = result[4].ToString();
        }
    }
    finally { }
}

```

В приведённом выше примере через 1 секунду после запуска подпрограммы происходит считывание строки, передаваемой по последовательному порту. Она записывается в переменную `outputLine`, после чего выводится в отладочную

консоль (рисунок 6). Строка имеет вид **A12B34C56D78E90**, где буквы – идентификаторы потенциометров, цифры – углы поворота.

Далее эта строка парсится, и результат записывается в массив. В массиве хранятся 5 значений типа `float` для углов поворота каждого из потенциометров, что соответствует поворотам 5 сервоприводов. Эти числа выводятся на интерфейс.

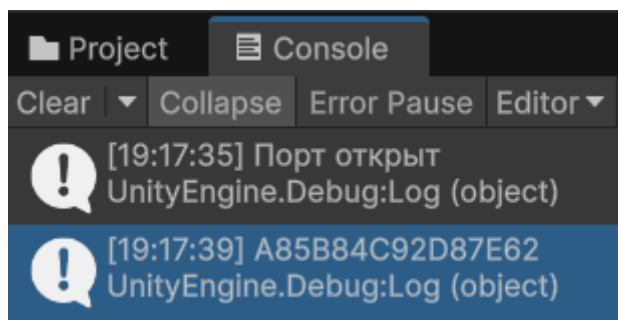


Рисунок 6. Вывод данных в консоль отладки Unity

Скрипт для парсинга значений углов из строки предлагается написать самостоятельно.

```
float[] ParseString(string data) // Парсинг углов из строки
```

...

В скрипте обязательно нужно добавить проверку, что строка начинается с буквы "A" – это защитит от неполноты данных, т.е. случаев, когда парсер начнёт работать с середины строки, из-за чего часть углов примут значение 0 или "n".

Определить, какой символ в строке следующий поможет механизм проверки регулярных выражений **Regex**. Требуемое регулярное выражение: буква + число.

```
Regex.Matches(data, @"[A-E](\d+)");
```

Если количество пар «буква+число» не равно 5, значит парсер обработал только часть строки, т.е. часть данных была потеряна при передаче. Следовательно, нужно прекратить и начать заново.

На рисунке 7 приведён пример интерфейса, куда выводятся значения углов поворота потенциометров. Кнопки позволяют открыть/закрыть последовательный порт, вручную очистить данные и получить текущие значения.

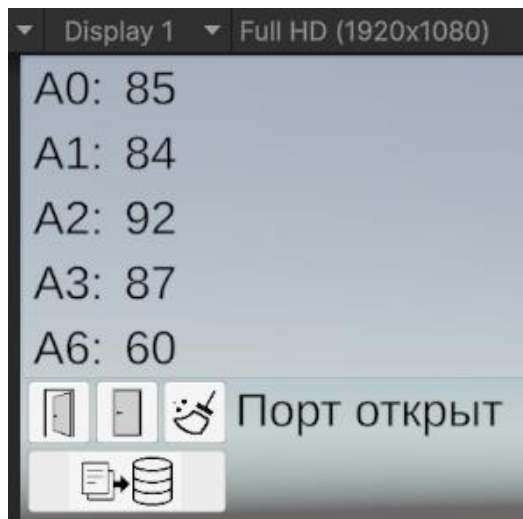


Рисунок 7. Вывод углов поворота на интерфейс

7. Поворот модели манипулятора в Unity

После того, как значения углов были приняты в Unity и отображены на интерфейсе, их можно использовать для поворота соответствующих игровых объектов.

Для осуществления поворота создайте новый скрипт **RotateServo.cs**.

В этом скрипте необходимо сперва определить трансформы шарниров, чтобы их можно было вращать на полученные от микроконтроллера углы.

```
public Transform arm;           // Манипулятор целиком
public Transform shoulder;      // Плечо
public Transform forearm;       // Предплечье
public Transform hand;          // Кисть
public Transform leftFinger;     // Левый палец
public Transform rightFinger;    // Правый палец
```

Поскольку углы сейчас хранятся в текстовом виде (и выводятся на экран как на рисунке 7), то перед выполнением поворота их понадобится перевести в целые числа.

```
int armAngle; int shoulderAngle; int forearmAngle; int handAngle; int
leftFingerAngle; int rightFingerAngle;
```

Также понадобится указать ссылки на скрипты ManualControl и GetSerialData:

```
public ManualControl mc;
public GetSerialData gsd;
```


Первый потребуется для получения локальных осей вращения шарниров, а второй – для получения текстовых данных об углах с микроконтроллера.

Воспользуемся методом Update для непрерывного получения данных об углах поворотов. В данном примере сделано так, что при отсутствии данных с микроконтроллера или при принудительной очистке данных в строку записывается символ «n». Поэтому понадобится добавить проверку: парсить углы только если значение в строке не «n»:

```
void Update()
{
    if (gsd.a0.text != "n")
    {
        armAngle = int.Parse(gsd.a0.text);
        Debug.Log("Угол руки: " + armAngle);
        shoulderAngle = int.Parse(gsd.a1.text);
        Debug.Log("Угол плеча: " + shoulderAngle);
        ...
    }
}

// a0 – A
// a1 – B
// a2 – C
// a3 – D
// a6 – E
```

Важное замечание: центральные позиции четырёх потенциометров, отвечающих за повороты манипулятора, соответствуют углу 90°, а центральная позиция потенциометра захвата соответствует углу 62,5°.

По этой причине просто так отправлять углы в метод Quaternion.Euler нельзя, иначе модель повернётся на 90° из начального положения. Поэтому для установки правильного начального положения достаточно вычесть 180 из значения поворота **arm**, вычесть 90 из поворота **shoulder** и **forearm**, а также вычесть 90 из значения поворота пальцев.

В данном примере для удобства и наглядности используется кнопка «Set Angles», при нажатии на которую виртуальный двойник поворачивается на углы, принятые с микроконтроллера (рисунок 8). Вам предлагается реализовать режим

вращения, при активации которого (например, с помощью переключателя UI >> Toggle) углы будут передаваться и обрабатываться непрерывно.

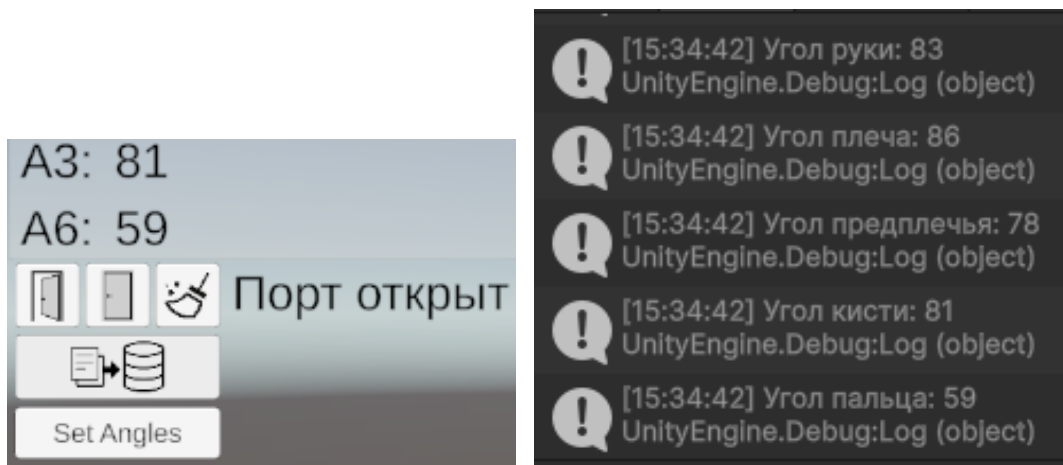


Рисунок 8. Обработка принятых углов

Само вращение производится методом `Quaternion.Euler` с указанием локальной оси вращения шарнира (из скрипта `ManualControl`):

```
arm.transform.localRotation = Quaternion.Euler(0f, 0f, armAngle - 180);
```

Узнать внутреннюю ось конкретного шарнира можно через `Debug`:

```
Debug.Log("Ось руки: " + mc.arm.rotationAxis);
```

...

Ось руки: (0.00, 0.00, 1.00)

Ось плеча: (1.00, 0.00, 0.00)

Ось предплечья: (1.00, 0.00, 0.00)

Ось кисти: (0.00, 0.00, 1.00)

Результат обработки поворотов показан на рисунке 9.

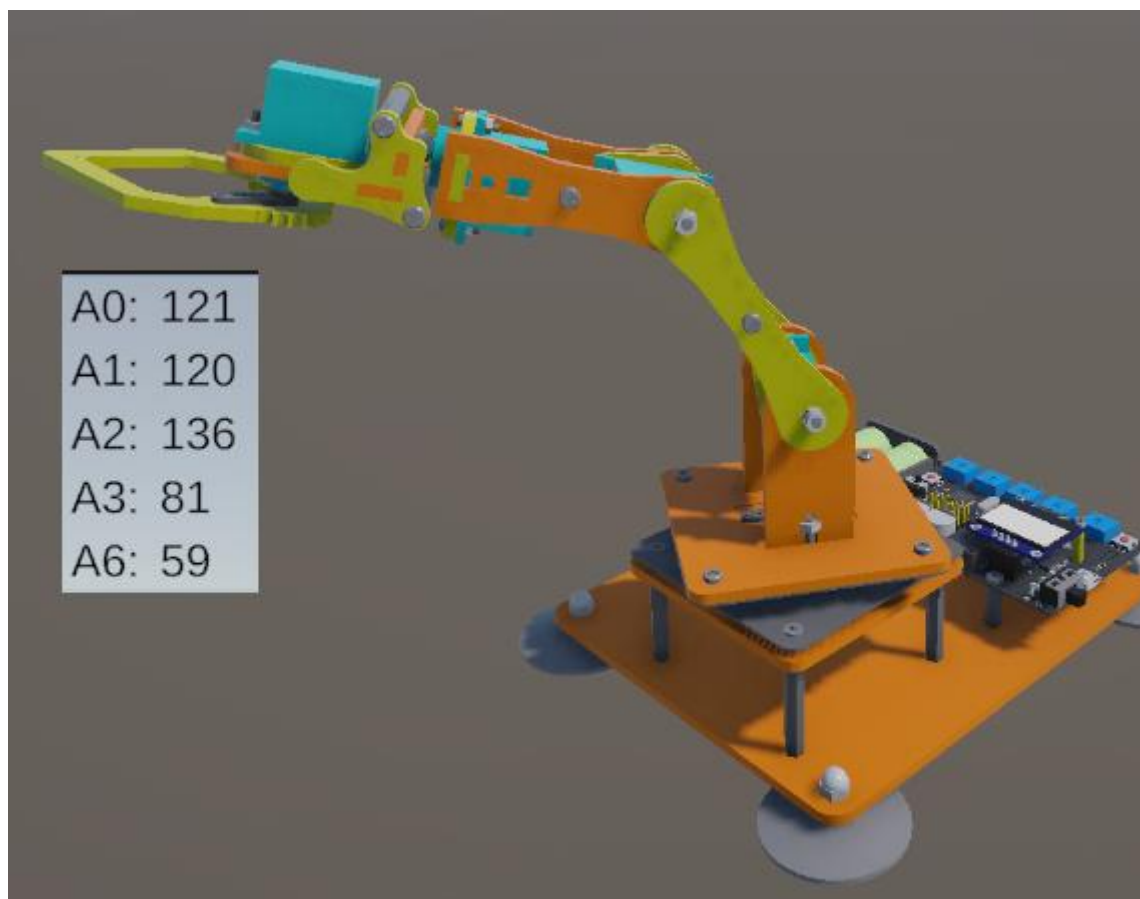


Рисунок 9. Повороты виртуального двойника

❖ Модификация скрипта **GetSerialData** в Unity

Изначально скрипт «GetSerialData.cs» считывал одну строку по нажатию на кнопку на интерфейсе. Такой способ работает для однократного получения углов поворота. Однако для вращения модели манипулятора в интерактивном режиме требуется, чтобы данные считывались и обрабатывались постоянно.

Потребуется модификация сопрограммы. Замените сопрограмму **DelayedAction** из примера выше на новую сопрограмму:

```
private IEnumerator ContinuousReading()
{
    while (true)
    {
        ...
        yield return null; // Ожидание следующего кадра
    }
}
```

Внутри неё находится цикл `while`, а в конце обозначено ожидание следующего кадра – `yield return null`. Зачем нужно это изменение? Изначально метод `GetData()` просто запускал сопрограму один раз. Она читала строку и выходила. Но чтобы данные обновлялись автоматически с некоторым интервалом, нужно, чтобы сопрограмма работала в цикле.

Метод `serialPort.ReadLine()` ждёт конца строки (;) и возвращает строку только один раз. Если данных больше нет, то следующая попытка чтения сразу же выдаст `TimeoutException` и вернёт `null`. Чтобы Unity постоянно получал данные, не нужно блокировать `ReadLine()`. Вместо этого нужно читать все данные, что приходят по последовательному порту, и аккумулировать строки.

```
// Если есть данные в порту
if (serialPort.BytesToRead > 0)
{
    // Чтение всего, что приходит с порта
    serialBuffer += serialPort.ReadExisting();
    // Разделение строки по символу конца строки ';'
    string[] lines = serialBuffer.Split(';');
    // Последний фрагмент остаётся в буфере
    serialBuffer = lines[lines.Length - 1];
    for (int i = 0; i < lines.Length - 1; i++)
    {
        string line = lines[i].Trim(); // Убрать пробелы и табуляцию в строке
        if (string.IsNullOrEmpty(line)) continue; // Пропустить пустую строку
    }
    ...
}
```

Здесь следует добавить код записи данных в массив, парсинга строки и вывода углов на интерфейс

Примечание: чтобы данный код работал, символ конца строки должен быть ";", а не "\n" !

Важную роль играет сохранение фрагмента в буфере. Допустим, по последовательному порту пришла строка:

A80B92C92D92E62;A81B93C93D93E63;A82B94C94

Как видно, она неполная – не хватает части символов и символа конца строки. Операция `Split(;;)` разделяет одну большую строку на несколько строк, оканчивающихся символом ";". Получается:

A80B92C92D92E62

A81B93C93D93E63

A82B94C94

Последнюю (неполная) строку не парсится сразу, потому что она ещё не закончена. Нужно подождать, пока придут недостающие данные. И когда Arduino присылает остаток «D94E64;», то `ReadExisting()` добавляет его к буферу. Символ конца строки удаляется, а пустая строка пропускается.

Данный цикл должен хорошо работать, если `serialPort.ReadTimeout = 1000`, а в Arduino задержка между отправками сообщений – `delay(500)`.

Используя полученные значения углов, реализуйте повороты шарниров манипулятора в реальном времени.

8. Общая информация о сервоприводах

Драйвер сервопривода на плате Adept Arm отправляет ШИМ-сигнал, который затем обрабатывается контроллером для расчёта направления вращения. Данные о вращении передаются через редуктор на качельку, которая жёстко соединена с корпусной деталью манипулятора. В то же время детектор положения возвращает сигнал о текущем положении, чтобы определить, была ли достигнута заданная позиция или нет. На рисунке 10 показана связь положения потенциометра с углом поворота и шириной генерируемого импульса.

Для управления вращением сервопривода в Arduino используется функция `write()`. Для стандартных сервоприводов (180°) функция выполняет поворот на соответствующую угловую позицию в абсолютном значении. Для 360-градусных сервоприводов функция `write()` задаёт не угол, а скорость вращения. А именно:

- 0 – полная скорость вращения в одном направлении;
- 180 – полная скорость вращения в противоположном направлении;
- 90 – стационарное положение.

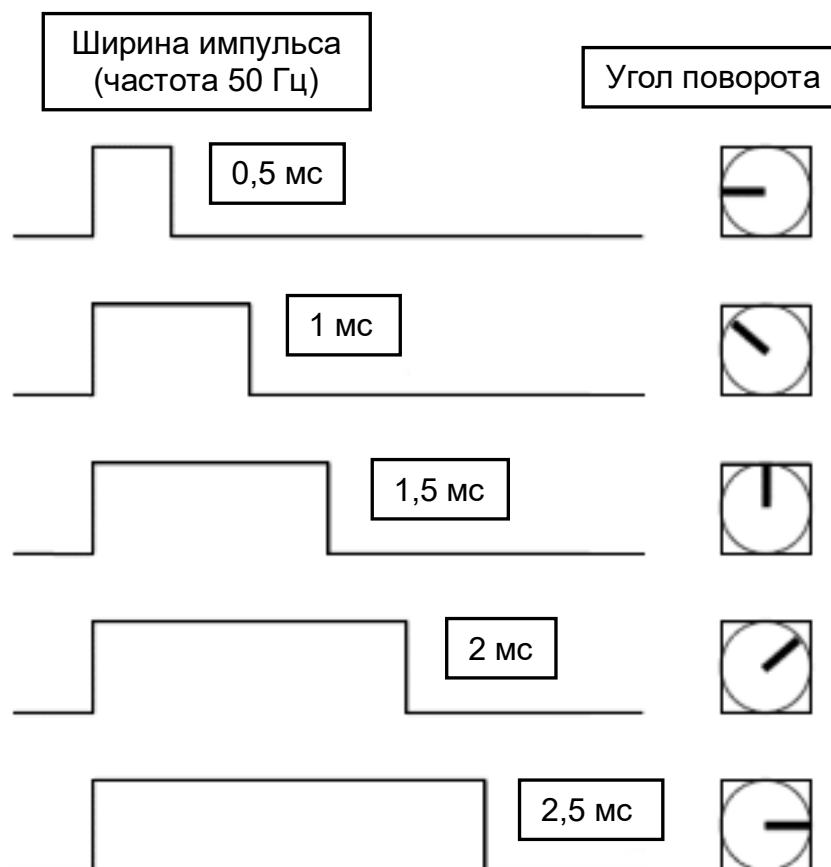


Рисунок 10. Зависимость углов поворота от ширины импульса

Типовой код приведён в скетче «**servo.ino**», расположенном по пути:

..\Adept_Robotic_Arm_Kit\Code\2. servo\servo

Например, для вращения руки относительно вертикальной оси (Z) код будет выглядеть следующим образом:

```
myservo.attach(9); // закрепить сервопривод на 9 контакте за объектом servo
myservo.write(0); // поворот сервопривода в начальное положение (0)
```

9. Формирование строки с углами в Unity

В ЛР №1 был реализован интерфейс для управления моделью робота в Unity при помощи клавиатуры/мыши и экранных кнопок.

Текущая задача:

- получить в реальном времени углы поворота каждого шарнира;
- сформировать строку, где будут указаны номер шарнира и угол в формате [букваЧисло];

- передать строку на микроконтроллер;
- в коде прошивки распарсить строку, получив из неё углы для каждого конкретного шарнира;
- перевести полученные углы в целочисленный формат [0..180];
- повернуть сервоприводы на полученные значения.

В классе шарнира `Hinge` есть переменная `currentAngle`, которая хранит текущий локальный угол поворота данного шарнира. Для получения строки достаточно преобразовать её в целочисленный чип `int` (отбросить дробную часть) и записать в переменную `rotationsString`, добавив перед числом букву, соответствующую шарниру. Например:

- для руки целиком (**arm**) – буква **A**;
- для плеча (**shoulder**) – буква **B**;
- для предплечья (**forearm**) – буква **C**;
- для кисти (**hand**) – буква **D**;
- для пальца (**leftFinger**) – буква **E**;

Введём дополнительные переменные для записи текущего угла в `int`:

```
private int armAngleData;
private int shoulderAngleData;
private int forearmAngleData;
private int handAngleData;
private int fingerAngleData;
```

В методе `Update()` нужно получить текущие углы и перевести их в `int`, после чего сформировать строку:

```
armAngleData = (int)arm.currentAngle;
...
rotationsString = "A"+armAngleData+"B"+shoulderAngleData+...
Debug.Log(rotationsString);
```

При ручном вращении робота (рисунок 11) получаем строку вида, например:

A-27B-21C55D-52E-36

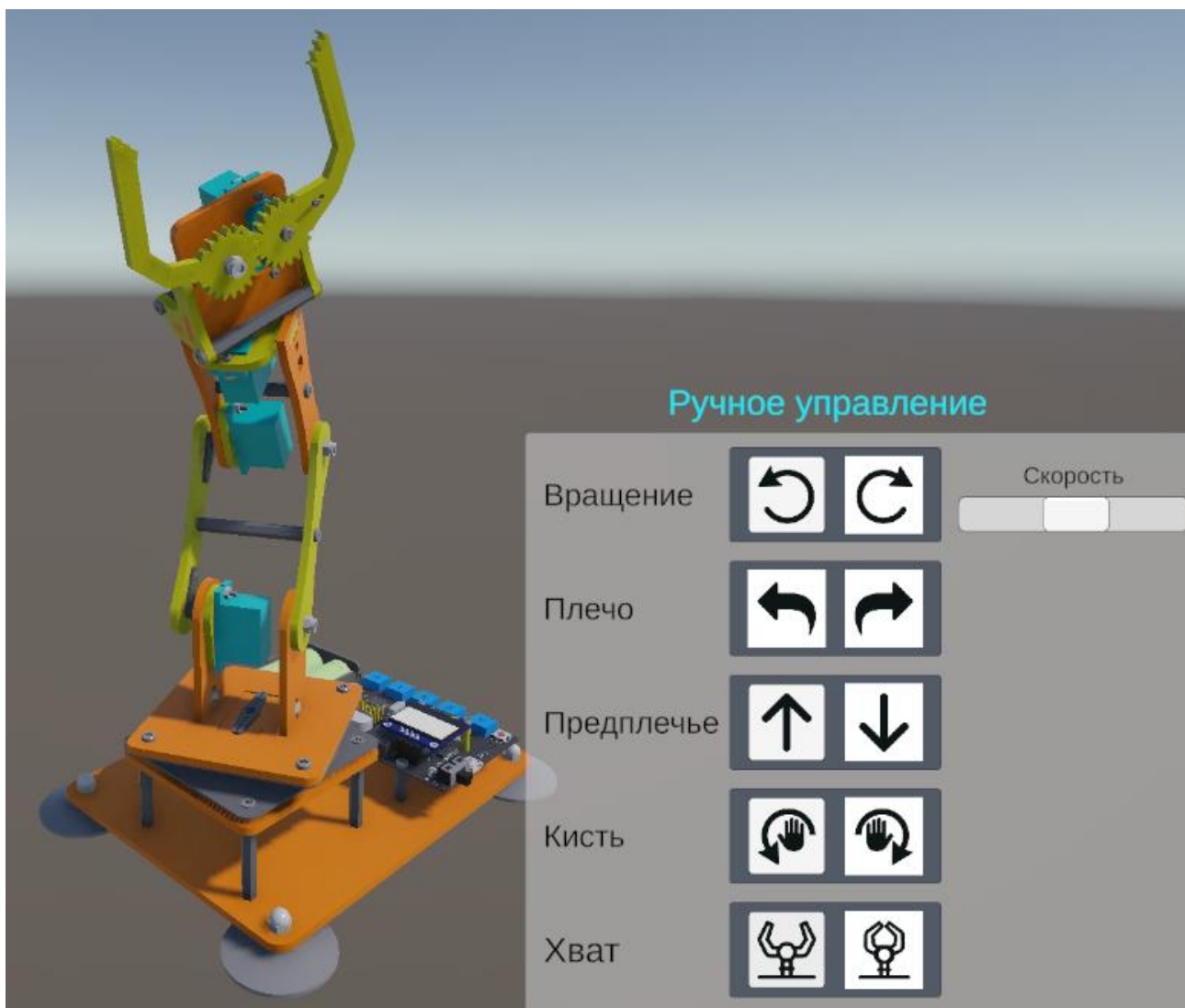


Рисунок 11. Ручное вращение робота

Начальное положение сервоприводов – 90° . Они специально калибруются в 90, чтобы иметь возможность поворачиваться как в одну, так и в другую сторону. В скетче Arduino для этого добавлены строки кода:

```
int dataServo1 = 90; // Калибровка серв в 90°
int dataServo2 = 90;
int dataServo3 = 90;
int dataServo4 = 90;
int dataServo5 = 90;
```

Поэтому необходимо, чтобы Unity отдавал строку, где 90 – это будет начальное положение шарниров. То есть осталось перевести значения углов в положительные числа. Для этого достаточно просто сместить весь диапазон на 90, т.е. прибавить 90 к каждому числу.


```
armAngleData = (int)arm.currentAngle + 90;
shoulderAngleData = (int)shoulder.currentAngle + 90;
...
```

Теперь сразу после запуска приложения мы получаем строку:

```
A90B90C90D90E90
UnityEngine.Debug:Log (object)
ManualControl:Update () (at Assets/Scripts/ManualControl.cs:95)
```

Введём ограничения по углам поворота (в скрипте `ManualControl.cs`):

- для руки целиком (**arm**) – от -90° до 90°
- для плеча (**shoulder**) – от -45° до 90°
- для предплечья (**forearm**) – от 45° до 180°
- для кисти (**hand**) – от -90° до 90°
- для левого пальца (**leftFinger**) – от -55° до 0°
- для правого пальца (**rightFinger**) – от 0° до 55°

10. Отправка углов поворота из Unity на сервоприводы

После формирования строки запишем её в глобальную переменную и отправим по Serial-интерфейсу при помощи скрипта `GetSerialData.cs`.

Сделайте переменную `rotationsString` глобальной.

Добавьте в код `IEnumerator SendAngles()`. Пусть он вызывается с задержкой в 1 секунду через `StartCoroutine(SendAngles());` в методе `Start()`.

```
IEnumerator SendAngles()
{
    while (true)
    {
        if (serialPort.IsOpen)
        {
            string _data = ManualControl.rotationsString;
            serialPort.WriteLine(_data);
            Debug.Log("Отправлена строка: " + _data);
        }
        yield return new WaitForSeconds(1f); // Интервал 1 секунда
    }
}
```

Если просто добавить запись в строку в метод `Update()`, то последовательный порт окажется перегружен данными.

Перейдём в среду Arduino IDE.

❖ **1 этап** – принять строку и вывести её на OLED-дисплей, чтобы убедиться в работоспособности Serial-интерфейса и правильности формата данных. Инициализируйте дисплей, после чего в функции `loop()` примите строку и отобразите её на дисплее (рисунок 12):

```
void loop()
{
  if (Serial.available() > 0) {
    String data = Serial.readStringUntil(';'); // Прочитать строку до ;
    display.clearDisplay();
    display.setCursor(0, 0); // Установить курсор
    display.setTextSize(2); // Сделать шрифт крупнее
    display.print(data); // Вывести данные
    display.display();
    delay(1000); // Задержка 1 сек
  }
}
```



Рисунок 12. Принятая из Unity строка с углами поворота

Обратите внимание, программа сама отрезала символ конца строки «;».

❖ **2 этап** – извлечь из полученной строки числа и записать их в переменные `val1`, `val2`, `val3`, `val4`, `val5`, соответствующие пяти сервоприводам. Для этого понадобится написать парсер. Для парсинга в C# в Unity мы использовали регулярные выражения `Regex`, однако в стандартной библиотеке Arduino (AVR) нет реализации `<regex>`, т.к. обработка регулярных выражений требует большого

объёма оперативной памяти и процессорного времени. Поэтому здесь будет применяться другой подход.

Пусть парсер называется `parseData` и ищет в строке букву и число после неё.

```
int parseData(String str, char ch) {
    int pos = str.indexOf(ch);
    if (pos == -1) return 0; // Буква не найдена

    // Начинаем с символа после буквы
    int start = pos + 1;
    int value = 0;

    // Считываем все цифры подряд
    while (start < str.length() && isDigit(str.charAt(start))) {
        value = value * 10 + (str.charAt(start) - '0');
        start++;
    }
    return value;
}
```

Метод `indexOf` ищет позицию буквы в строке. Если буквы нет, возвращается значение `-1` и выполнение прекращается.

Указатель `start` выставляется на следующий символ после буквы. Создаётся переменная `value`, куда будет записано число.

Далее следует самый важный алгоритм формирования чисел. Его идея – использовать коды кодировки ASCII (в которой хранятся символы в памяти) для цифр и вычитать один код из другого для нахождения числового значения. Например, если из кода «50» (цифра 2) вычесть код «48» (цифра 0), получится числовое значение 2.

Каждый раз при нахождении новой цифры накопленное число умножается на 10, чтобы осуществить сдвиг влево (в разряд десятков или соток) и освободить место для следующей цифры в разряде единиц. Алгоритм выполняется, пока не конец строки, и текущий символ является цифрой (`isDigit`).

Осталось вызвать метод, указав желаемую букву, после которой требуется получить число.

```
val1 = parseData(data, 'A');  
...
```

В момент открытия Serial-порта по нему начинают приходить нули, которые могут быть неверно интерпретированы как углы. Самый простой способ избежать этого – добавить проверку: перемещение выполняется только в том случае, если переменные val не равны 0.

```
if (val1 != 0) servoA.write(val1);  
if (val2 != 0) servoB.write(val2);  
if (val3 != 0) servoC.write(val3);  
if (val4 != 0) servoD.write(val4);  
if (val5 != 0) servoE.write(val5);  
delay(1000); // Задержка 1 сек
```

На рисунках 13 и 14 показаны результаты работы – движение реального робота повторяют движение его виртуального двойника из Unity.

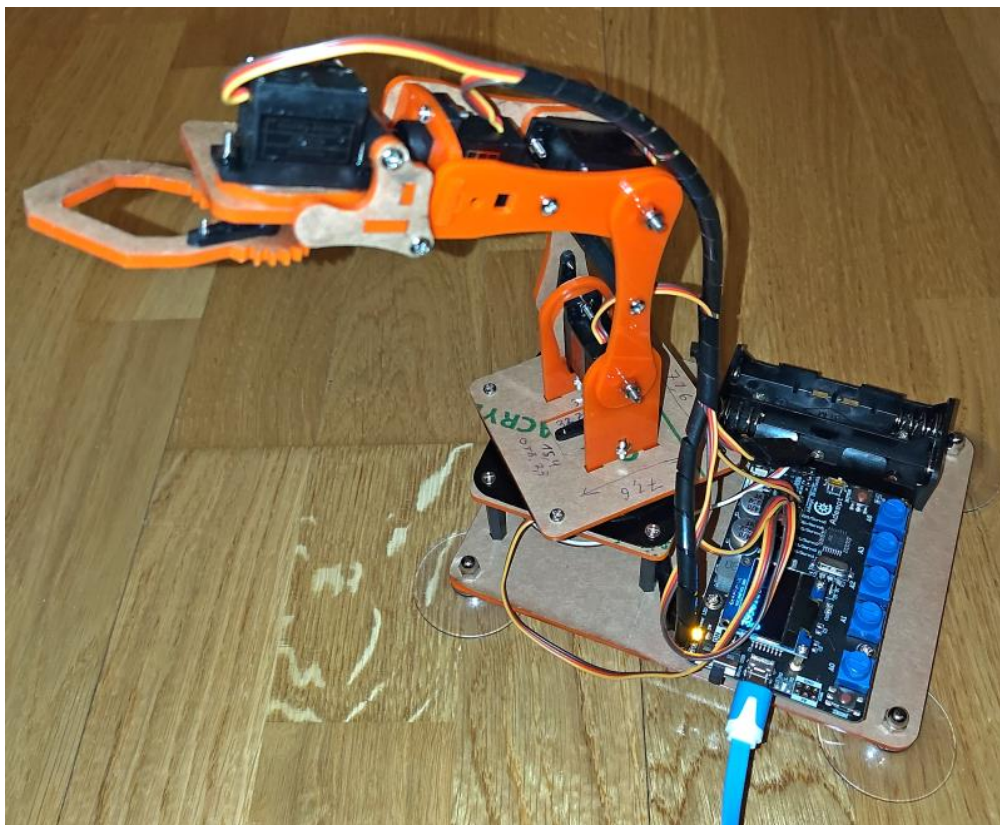


Рисунок 13. Поворот робота-манипулятора



Рисунок 14. Поворот виртуального двойника

11. Содержание и оформление отчёта

Отчёт о лабораторной работе оформляется в соответствии с требованиями ГОСТ 7.32 – 2017, которые доступны по адресу: <https://guap.ru/regdocs/docs/uch>

Содержание отчёта о лабораторной работе:

- 1) Титульный лист.
- 2) Цель работы.
- 3) Вариант задания (в какую сторону будет идти управление).
- 4) Как происходит работа с Serial интерфейсом в Unity.
- 5) Как происходит работа с сервоприводами в Arduino IDE.
- 6) Как реализовать повороты виртуального двойника на основе значений, принятых с микроконтроллера: алгоритм, скриншоты, листинг кода.
- 7) Как реализовать повороты робота на основе значений, принятых из Unity: алгоритм, скриншоты, листинг кода.
- 8) Выводы о проделанной работе.